TP2 — TaskFlow devient interactif et navigable

Module 2 · État, useEffect, FlatList et React Navigation

Élément	Détail
Projet fil rouge	TaskFlow — application mobile de gestion de tâches, construite de TP en TP
Point de départ	Le projet TaskFlow livré à la fin du TP1 (tag Git tp1-fin).
Point d'arrivée	Une application interactive à 4 écrans avec onglets et pile de navigation.
Git	Créez une branche tp2 depuis le tag tp1-fin (git switch -c tp2 tp1-fin). À la fin : commit, puis git tag tp2-fin.

Où se situe ce TP dans le projet

TP	Étape	Ce que l'application gagne
TP1	Mise en place	Projet Expo, structure src/, composant TacheCard, écran d'accueil statique
TP2 (ce TP)	Interactif & navigable	State des tâches (ajout, bascule, suppression), FlatList, navigation Stack + Tab
TP3	Base de données	Base MySQL taskflow : 3 tables, données de test, requêtes CRUD, JOIN, agrégations
TP4	API & connexion	API Express CRUD, service axios, écrans branchés sur l'API (loading / error / success)
TP5	Application connectée	Lieu (GPS), photo (caméra), rappels (notifications), secousse (capteurs), synchronisation temps réel (WebSocket)
TP6	Sécurité & finalisation	bcrypt, JWT, SecureStore, navigation conditionnelle,

© 2026 Caplogy. All rights reserved.

		finitions, soutenance
TP7	Optimisation (bonus)	Mémoïsation, pagination, recherche avec debounce, index SQL, mesures

0. Où en est TaskFlow

Vous reprenez exactement le projet livré à la fin du TP1 : le projet Expo TaskFlow, son composant TacheCard, et son écran d'accueil affichant un tableau statique de tâches. Ne créez pas un nouveau projet : vous continuez celui-ci.

> **i Information**
>
> Si votre TP1 n'est pas terminé ou fonctionnel, reprenez d'abord la correction du Module 1 avant de commencer ce TP.

1. Objectifs pédagogiques du TP2

- Convertir des données statiques en state modifiable avec useState.
- Implémenter l'ajout, la validation et la suppression de tâches.
- Remplacer .map() par FlatList pour l'affichage de la liste.
- Installer et configurer React Navigation (Stack + Tab).
- Créer une navigation entre 4 écrans avec transmission de paramètres.

2. Partie 1 — Rendre la liste de tâches interactive

2.1 Convertir le tableau en state

Dans AccueilScreen.js, remplacez le tableau TACHES en dur par un state initialisé avec les mêmes données :

AccueilScreen.js (extrait)

```
const [taches, setTaches] = useState([
  { id: '1', titre: 'Créer la maquette', categorie: 'Design', terminee: true },
  // ... les autres tâches du Module 1
]);
```

2.2 Cocher une tâche comme terminée

© 2026 Caplogy. All rights reserved.

Ajoutez une fonction qui bascule le champ terminee d'une tâche précise, sans muter le tableau existant (revoir la Partie 1 du cours si besoin).

- Écrire une fonction basculerTache(id) qui utilise .map().
- Rendre chaque TacheCard cliquable (Pressable englobant) pour déclencher cette fonction.

2.3 Supprimer une tâche

Ajoutez un bouton ou une icône "Supprimer" sur chaque tâche, qui retire l'élément du state avec .filter().

3. Partie 2 — Ajouter une nouvelle tâche

Créez un nouvel écran src/screens/AjouterScreen.js contenant :

- Un TextInput contrôlé pour le titre de la tâche.
- Un second TextInput (ou une petite liste de boutons) pour choisir la catégorie.
- Un bouton "Ajouter" qui crée une nouvelle tâche et revient à l'écran d'accueil.

> 💡 **Indice**
>
> Le state 'taches' doit rester dans un seul endroit (par exemple AccueilScreen, ou un fichier séparé) afin qu'AjouterScreen puisse l'utiliser via une fonction transmise en paramètre de navigation ou un contexte. Pour ce TP, la solution la plus simple consiste à passer une fonction onAjouter via les params, ou à définir le state au niveau du composant racine et à le transmettre en props aux écrans.

4. Partie 3 — Passer à FlatList

Remplacez le .map() de AccueilScreen par un composant FlatList :

- data doit recevoir le state taches.
- keyExtractor doit utiliser l'id de chaque tâche.
- renderItem doit afficher un TacheCard pour chaque tâche.
- Ajoutez un ListEmptyComponent affichant un message quand la liste est vide.

5. Partie 4 — Mettre en place la navigation

© 2026 Caplogy. All rights reserved.

5.1 Installation

terminal

```
npm install @react-navigation/native
npx expo install react-native-screens react-native-safe-area-context
npm install @react-navigation/native-stack
npm install @react-navigation/bottom-tabs
```

5.2 Les écrans à créer

Écran	Fichier	Rôle
Accueil	src/screens/AccueilScreen.js	Liste des tâches (FlatList)
Ajouter	src/screens/AjouterScreen.js	Formulaire de nouvelle tâche
Detail	src/screens/DetailScreen.js	Détail d'une tâche sélectionnée
Profil	src/screens/ProfilScreen.js	Écran simple avec vos informations

5.3 Architecture de navigation attendue

Mettez en place un Tab Navigator avec deux onglets : "Tâches" (contenant un Stack Navigator Accueil → Ajouter → Detail) et "Profil" (écran simple). Reportez-vous au schéma du cours (Partie 5) pour l'architecture imbriquée exacte.

- Créer src/navigation/AccueilStack.js avec le Stack Navigator (Accueil, Ajouter, Detail).
- Créer src/navigation/AppNavigator.js avec le Tab Navigator (Tâches, Profil).
- Mettre à jour App.js pour n'afficher que <AppNavigator /> dans un <NavigationContainer>.

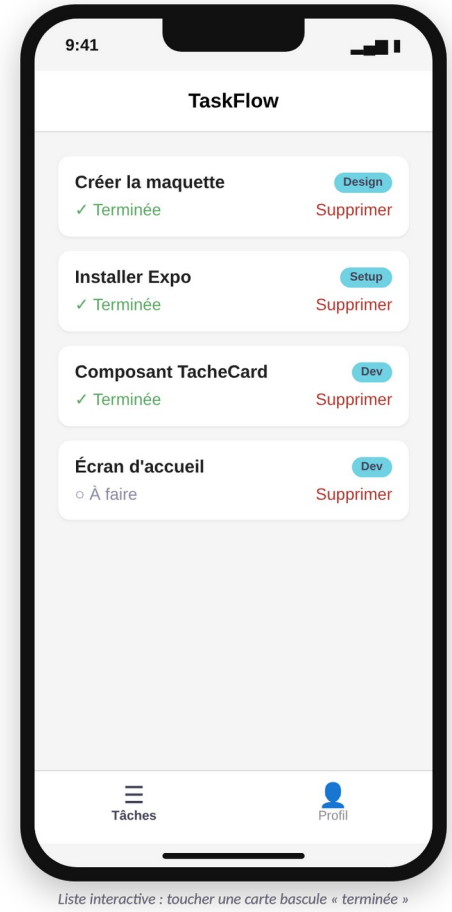
5.4 Navigation vers le détail

Depuis AccueilScreen, en appuyant sur une TacheCard, naviguez vers DetailScreen en transmettant l'id et le titre de la tâche via route.params. DetailScreen doit afficher ces informations et proposer un bouton retour.

Rendu attendu

© 2026 Caplogy. All rights reserved.

Les captures ci-dessous montrent le résultat à obtenir. Les écrans de téléphone sont des maquettes fidèles aux styles du cours ; les captures de terminal proviennent d'exécutions réelles.

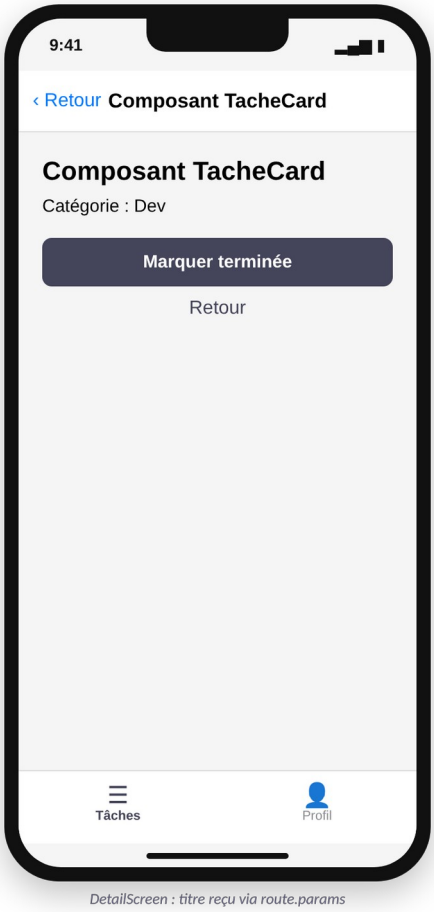


Liste interactive (Accueil)

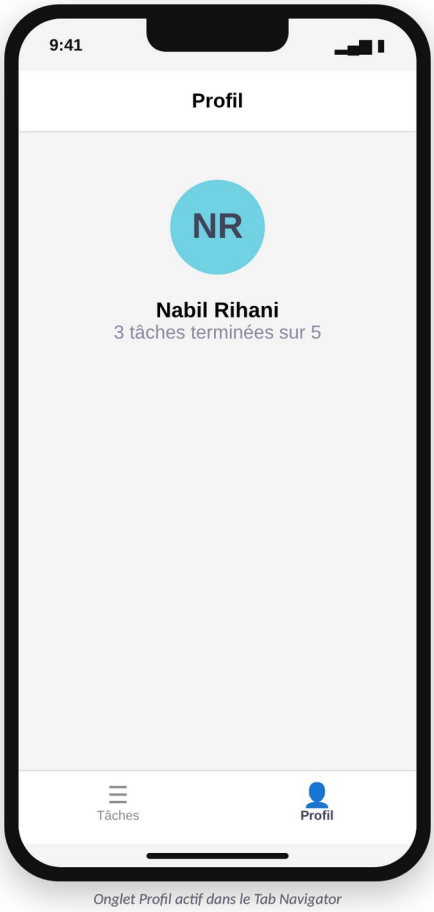


Écran d'ajout

© 2026 Caplogy. All rights reserved.



Écran de détail



Onglet Profil

6. Livrables attendus

Critère	Validé
Le state 'taches' remplace le tableau statique du Module 1	☐
Ajout, validation et suppression de tâches fonctionnels	☐
FlatList utilisée à la place de .map(), avec ListEmptyComponent	☐
React Navigation installé et configuré (Stack + Tab)	☐
4 écrans créés : Accueil, Ajouter, Detail, Profil	☐
Navigation vers Detail avec transmission de	☐

© 2026 Caplogy. All rights reserved.

paramètres (id, titre)	
Un nouveau commit Git du projet	☐

7. Critères d'évaluation

Critère	Pondération
Interactivité de la liste (ajout/coche/suppression)	30%
Utilisation correcte de FlatList	15%
Mise en place de la navigation Stack + Tab	30%
Transmission correcte des paramètres de navigation	15%
Qualité générale du code	10%

i Information

Vers le Module 3

Une fois ce TP terminé, TaskFlow gère ses données uniquement en mémoire (elles disparaissent à chaque redémarrage de l'app). Le Module 3 introduira une vraie base de données relationnelle pour stocker ces informations durablement.

Avant de passer au TP suivant

- Exécuter l’application de bout en bout et vérifier chaque point de la grille.
- git add . && git commit -m "TP2 terminé" && git tag tp2-fin
- Le TP3 repartira exactement de ce tag : ne supprimez rien.

© 2026 Caplogy. All rights reserved.